

# Procedural Texture Baker

Node-graph texture generator with GPU-accelerated live preview and one-click PBR map export for albedo, normal, roughness, and AO.

|      |      |      |      |
|------|------|------|------|
| Rust | wgpu | WASM | egui |
|------|------|------|------|

| VERSION | STATUS   | MODE        | LICENSE | TARGETS      | UPDATED  |
|---------|----------|-------------|---------|--------------|----------|
| 2.0.0   | Complete | Maintenance | MIT     | Native, WASM | Sep 2026 |

## 1. Overview

The Procedural Texture Baker generates tileable material textures from a node graph. Noise, pattern, warp, filter, and blend nodes are wired together, evaluated on the GPU, and baked out as a complete PBR set: albedo, normal, roughness, and ambient occlusion. The preview updates while the graph is edited, at full material resolution rather than on a proxy.

The whole thing compiles to a single Rust binary with no runtime dependencies, and the same source builds to WebAssembly for a browser version that shares the graph format. Graphs are plain text and diff cleanly, which is what allows a material library to live in version control alongside the project that uses it.

## 2. Evaluation pipeline

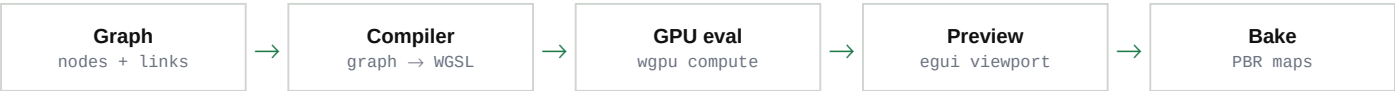


Figure 1 — The graph compiles to one compute kernel per output map; preview and bake differ only in resolution and tiling.

Compiling the graph into a single kernel, rather than dispatching one pass per node, is what keeps the preview interactive. Identical subtrees are deduplicated during compilation, so a noise node feeding four downstream branches is evaluated once and its result reused across all of them within the same dispatch.

## 3. Components

|             |                                                                                             |
|-------------|---------------------------------------------------------------------------------------------|
| Graph model | Typed nodes and links with cycle detection; serialised as human-readable text.              |
| Compiler    | Topological lowering to WGSL with common-subexpression elimination and constant folding.    |
| GPU runtime | wgpu compute dispatch over Vulkan, Metal, DX12, and WebGPU from one code path.              |
| Preview     | egui viewport with a lit material sphere and plane, tiling overlay, and per-map inspection. |
| Baker       | Tiled evaluation for large outputs, mip generation, and PNG, EXR, or KTX2 export.           |

## 4. Node categories

| CATEGORY   | EXAMPLES                                      | NOTES                                    |
|------------|-----------------------------------------------|------------------------------------------|
| Generators | Perlin, Worley, Voronoi, gradient, shape      | All seamlessly tileable by construction. |
| Warps      | Domain warp, flow, directional stretch        | Composable; nested warps supported.      |
| Filters    | Blur, curve, levels, edge, normal-from-height | Operate per channel or on luminance.     |
| Outputs    | Albedo, normal, roughness, AO, height         | Each output compiles its own kernel.     |

## 5. Graph format

```
graph "rusted-plate" {
  version = 2
  resolution = 2048
  seed = 0x51A3

  node base      : worley { scale = 12.0, jitter = 0.8, metric = "euclidean" }
  node grain     : perlin { scale = 220.0, octaves = 4, gain = 0.45 }
  node warped    : warp  { input = base, by = grain, amount = 0.06 }
  node height    : levels { input = warped, lo = 0.18, hi = 0.92 }
  node rust      : curve  { input = grain, points = [[0,0.1],[0.6,0.7],[1,0.95]] }

  out albedo     = blend { a = "#5A4436", b = "#8C3A1B", mask = rust }
  out normal     = normal_from_height { input = height, strength = 1.4 }
  out roughness  = curve { input = rust, points = [[0,0.35],[1,0.85]] }
  out ao         = occlusion { input = height, radius = 8, samples = 24 }
}
```

Listing 1 — Graphs are text-first; the visual editor reads and writes this format directly.

## 6. Baking behaviour

- **Tiling.** Outputs above 4K are evaluated in tiles with an overlap margin sized to the widest filter kernel in the graph, so tile seams cannot appear in blurred or occlusion outputs.
- **Determinism.** The seed plus graph text fully determines the result on every backend. Cross-platform bakes are bit-identical for integer paths and within one ULP for floating point.
- **Formats.** PNG for 8-bit channels, EXR for height and float data, KTX2 with BC7 or ASTC compression for engine-ready delivery, all with mips generated during the bake pass.

## 7. Performance

| OPERATION             | TIME  | HARDWARE                        |
|-----------------------|-------|---------------------------------|
| Preview update, 1K    | 4 ms  | RTX 4060 laptop, 26-node graph. |
| Full bake, 4K, 4 maps | 1.9 s | Same graph and hardware.        |
| Full bake, 8K, 4 maps | 7.4 s | Tiled, 512 px overlap.          |
| WASM preview, 1K      | 31 ms | Chrome 129, WebGPU, same graph. |

## 8. Build and integration

- Native builds are a single static binary per platform; the WASM build ships as a page with no server component, sharing the graph parser and compiler with the desktop version.
- A headless CLI bakes a graph directly, which is how material libraries are regenerated in CI after a shared node or palette change.
- Requires a WebGPU-capable browser or a Vulkan, Metal, or DX12 device; there is no CPU fallback, by design, since the tool's interactivity depends on GPU evaluation.

## 9. Maintenance status

| AREA            | STATE  | COMMITMENT                                                 |
|-----------------|--------|------------------------------------------------------------|
| Feature work    | Closed | The 2.0 node set is final; no new node categories planned. |
| Bug fixes       | Active | Correctness and platform regressions fixed as reported.    |
| Backend support | Active | Tracks wgpu releases and browser WebGPU changes.           |